

Assignment Intelligence

Helping the floor manager pick the right designer, using what we already know

Team

Member	Expertise
Kavinraj Santhi Ilangovan Working alone	User interface development. I work as a developer at Madarth, which is where this problem happens. I can get at the workflow, the Basecamp data and the nine people listed further down. The hardest design question here is a screen question. I have to show a suggestion, the evidence behind it, and the times the system has nothing to say, without the whole thing looking like a league table of designers. Any screen showing a sorted list of names will be read that way, whatever the maths underneath is doing.

Problem Statement

How work moves through Madarth today

A client sends us a task. It goes to a Client Coordinator, who passes it to the floor manager. The floor manager picks a writer, then picks a designer. When the design is finished it goes back to the Client Coordinator, who either approves it or asks for changes. We allow up to three rounds of changes. After that it goes to the client, or straight onto social media. Six designers handle roughly 260 tasks a month between them.

What goes wrong

The floor manager picks the designer in a few seconds, from memory. He has a rough sense of who is good at what, built from the jobs he happens to remember. When that sense is wrong, the design comes back for changes and the job runs late.

The information he needs is already sitting in Basecamp. Every job we have done is in there, along with how many rounds of changes it took before it was approved. Nobody can read any of that at the moment they have to choose.

This looks like a scheduling problem at first. It is closer to a memory problem. Someone makes the same decision over and over, with worse information than the company already holds.

Why this is harder than running a database query

Basecamp closes a job when it is approved, so we always know when a job finished. The reason for each round of changes is a different matter. It is written in normal comments, in the shorthand people use at work. One comment can hold a correction, a client request and a note about timing, all in the same sentence.

That difference decides everything. If a client changes their mind, the designer redoes the work. If the system counts that as the designer getting it wrong, then a designer with fussy clients ends up looking like a designer who makes mistakes.

There is a second difficulty. Two Client Coordinators can read the same comment thread and disagree about what it means. So even the correct answer is a judgement call, and I have to measure how much they agree before I trust any of it.

What I expect to happen

If the floor manager can see, at the moment he chooses, how much rework each designer's similar past jobs needed, and can also see when there is too little history to say anything at all, then he will change some of what he currently believes about who suits what. He built those beliefs from memory rather than from the record, so some of them should turn out to be wrong.

Ideal Solution

What the system does

It reads a new job description and works out what kind of design it is. Then it shows the floor manager which designers have done similar work before, how that work went, and the actual jobs it is basing that on, so he can open them and check for himself.

It gives him a choice rather than an answer. One designer is the safe pick. Another is someone who could learn from this job, and the system says what the risk of choosing them is. When there is not enough history, it says so and suggests nobody. Saying nothing counts as a proper answer here.

What it will not do

- It will not use deadlines or timing data to pick a designer.
- It will not score designers or rank them against each other.
- It will not read client conversations from WhatsApp. Basecamp only.

The system looks at the work, not at the person. There is a reason for that. How much rework a job needed depends on how clear the brief was, how much the client changed their mind, and how strict that particular Client Coordinator is. The designer is only one part of it. Turning all of that into a score for a person would mean handing someone a confident number I cannot back up.

Why a model is needed at all

Reading a comment thread and telling a correction apart from a client request is a judgement. Simple rules get it wrong too often to build on. That is the part the model does, and I will measure how well it does it rather than assume it works. The counting and the maths that follow are ordinary code. I have left the model out of that part on purpose.

How I will check whether it works

First, I check the reading. I will take around 150 comment threads. The Client Coordinators label one set to teach the system. I keep a second set for testing my own changes as I go. I lock away 50 and open them only at the end. I will score each category on its own, because a decent average can hide a bad result on the one category that matters most.

Second, I check the labels themselves. Both Client Coordinators label the same 30 threads separately. If they do not agree with each other, my categories are unclear and I fix them before building anything on top.

Third, I ask the floor manager to write down what he believes about each designer before he sees any output from the system. That way I am testing something he predicted in advance, instead of asking him afterwards whether he liked what he saw.

Then I test it properly. The designer history comes from past Basecamp data. The suggestions get tested on real jobs as they arrive. I will compare the system against the floor manager choosing on his own, and

against a plain rule such as giving the job to whoever has the least on.

What I have to work with

Six designers, two Client Coordinators and one floor manager, all willing to take part. Around 260 jobs a month in Basecamp. I have access to the data, and written permission to publish what I find.

What this system will be bad at

- **One Coordinator, one project.** A Client Coordinator handles a project from start to finish. So the rework count describes a designer and that Coordinator together, not the designer on their own. I will only compare designers working under the same Coordinator.
- **Thin evidence for the learning suggestion.** Designers keep getting the same kind of work. So I have plenty of history on what they already do and almost none on what they have never been given. That is exactly what the learning suggestion needs, so it will be the weakest part.
- **Small numbers.** Six designers across many job types does not leave much per designer. The system will often have to admit it does not know.
- **Approved does not always mean good.** A Coordinator may approve weaker work when a deadline is close.
- **People know they are being watched.** Everyone taking part knows their work is being read, so they may behave differently while I am testing.

I would rather build something that admits when it does not know, and can show those refusals were right, than something that always has an answer it cannot explain.